

AI Resume Analyzer and Job Recommendation System

Major Project of AI — Project Report

Student: Nikhil D P

Batch: Artificial Intelligence June (A) 2026 Batch Program

[GitHub Repository: https://github.com/nikhilaaradhya3-cpu/ai-resume-analyzer.git](https://github.com/nikhilaaradhya3-cpu/ai-resume-analyzer.git)

[Live Demo: https://ai-resume-analyzer-nq7auhmtwaado5bbnscjyd.streamlit.app/](https://ai-resume-analyzer-nq7auhmtwaado5bbnscjyd.streamlit.app/)

1. Project Objective

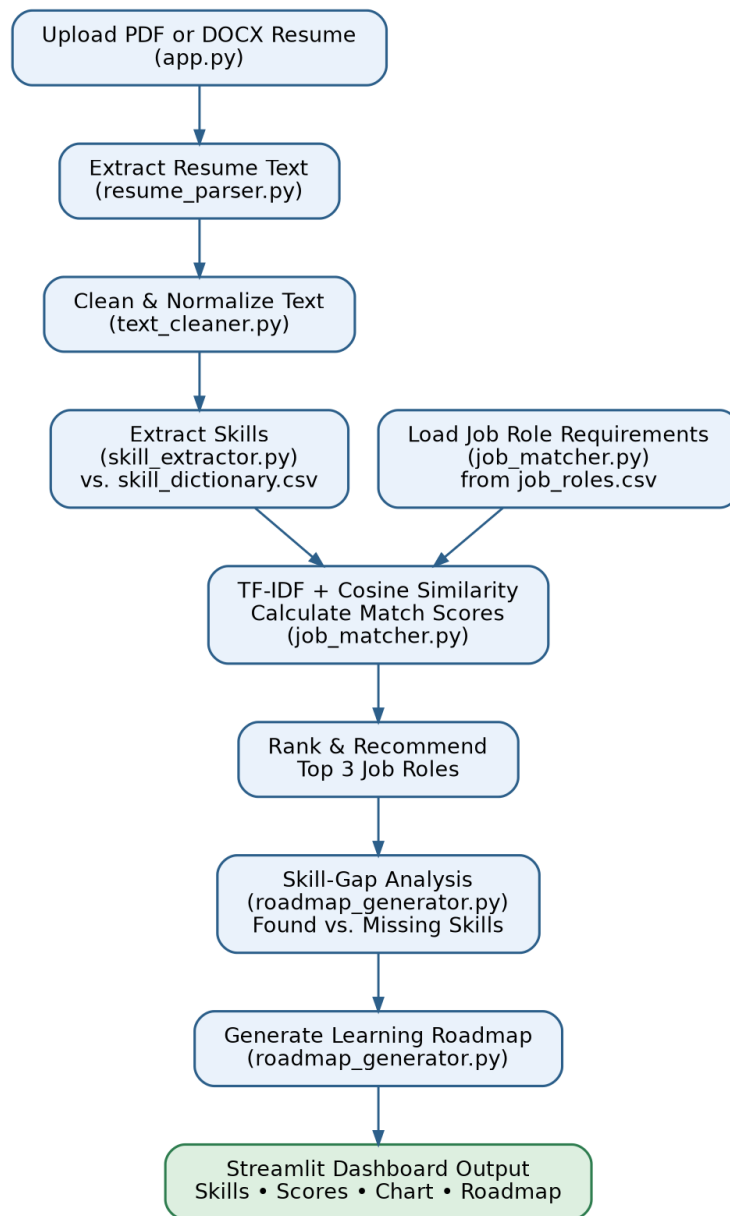
The goal of this project is to build an NLP-based application that helps students understand how well their resume matches selected job roles. The system provides a match score, lists important missing skills, and generates a simple, actionable learning roadmap.

2. Problem Statement

Students often do not know whether their resume contains the skills expected for a particular role, and recruiters receive many resumes and need a quick way to compare job-related information. This project creates an educational, skills-focused matching system that evaluates resumes based on skills, projects, education, and relevant experience only.

3. System Architecture and Workflow

The application follows a linear pipeline from resume upload to final recommendations, as shown below.



4. Tools and Technologies

Project Part	Tool / Technology
Programming language	Python
PDF resume extraction	pypdf
DOCX resume extraction	python-docx
Text cleaning	Python and regular expressions
Matching approach	TF-IDF and cosine similarity (scikit-learn)
Data handling	Pandas
Charts	Streamlit built-in bar chart

User interface	Streamlit
Deployment	Streamlit Community Cloud
Version control	Git and GitHub

5. Main Modules

Module 1: Resume Upload

Accepts PDF and DOCX files through a Streamlit file uploader. Files are processed in memory only and are not stored permanently, in line with the project's privacy requirements.

Module 2: Text Extraction and Cleaning

resume_parser.py extracts raw text from every PDF page or DOCX paragraph. text_cleaner.py lowercases the text, removes URLs/emails and unnecessary symbols, while retaining important technical symbols such as C++, C#, and .NET.

Module 3: Skill Extraction

skill_extractor.py searches the cleaned text against a controlled skill dictionary (skill_dictionary.csv) covering programming, database, machine learning, cloud, and tools categories, using regex-based keyword matching.

Module 4: Job Role Dataset

job_roles.csv defines five job roles (Data Analyst, Machine Learning Engineer, AI Engineer, NLP Engineer, Computer Vision Engineer) with their associated required skills.

Module 5: Matching and Recommendation

job_matcher.py converts the candidate's extracted skills and each role's requirements into TF-IDF vectors, computes cosine similarity, and ranks roles from highest to lowest match score.

Module 6: Skill-Gap Analysis and Roadmap

roadmap_generator.py compares extracted skills against the selected target role's requirements, lists found and missing skills, and generates a rule-based, week-by-week learning roadmap for the missing skills.

Module 7: Streamlit Dashboard

app.py ties all modules together: resume upload, target-role selection, extracted skills display, match-score metrics and chart, top-3 recommended roles, skill-gap breakdown, and the learning roadmap.

6. Testing and Evaluation

The application was tested against four anonymized sample resumes, each written to align with a specific target role. The actual output was produced by running the project's own extraction and matching code against each file.

Test Resume	Expected Top Role	Actual Top Role	Match Score
resume_data_analyst.docx	Data Analyst	Data Analyst	100.0%
resume_ml_engineer.docx	Machine Learning Engineer	Machine Learning Engineer	56.17%
resume_nlp_engineer.docx	NLP Engineer	NLP Engineer	78.28%
resume_cv_engineer.docx	Computer Vision Engineer	Computer Vision Engineer	81.11%

All four test resumes correctly returned their expected role as the top recommendation, confirming that the skill extraction and TF-IDF/cosine-similarity matching logic behave as intended.

7. Responsible AI Considerations

- The tool is intended for guidance only, not automatic hiring or rejection.
- It does not score gender, age, religion, nationality, photographs, marital status, or disability.
- It evaluates only job-related skills, education, projects, and relevant experience.
- Match scores are presented as estimates, not recruiter decisions.
- Uploaded resumes are processed in memory only and are not stored permanently.
- Missing keywords are not claimed to always mean missing ability.

8. Limitations and Future Work

The current version relies on keyword-based skill extraction and a manually curated dataset of five job roles and around twenty skills, which may miss synonyms or indirectly phrased skills. Planned improvements include semantic matching using Sentence Transformers, spaCy-based entity and phrase extraction, a larger and more dynamic job-role dataset, and optional LLM-generated resume feedback.

9. Final Deliverables

- Working Streamlit application (deployed link above)
- Complete source code (GitHub repository)
- Job-role and skill datasets (data/job_roles.csv, data/skill_dictionary.csv)
- Sample resumes with personal information removed (sample_resumes/)
- requirements.txt
- README with setup and usage instructions
- Architecture/workflow diagram (docs/architecture.png)
- Testing sheet (tests/test_cases.csv)
- GitHub repository
- Short project report and demonstration video